UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
BACHARELADO EM ENGENHARIA DE SOFTWARE
DISCIPLINA: MODELAGEM DE SOFTWARE

Carlos Eduardo Freire da Silva
Gabriel Henrique Lourenço Dias
Kayque Vicente Silva
Luiz Pedro Pereira Mendes Saraiva
Yury Nogueira de Oliveira Silva

SISTEMA DE AGENDAMENTO DE CONSULTAS MÉDICAS

Modelagem de Software — APS 1

GOIÂNIA

2026

Carlos Eduardo Freire da Silva
Gabriel Henrique Lourenço Dias
Kayque Vicente Silva
Luiz Pedro Pereira Mendes Saraiva
Yury Nogueira de Oliveira Silva

SISTEMA DE AGENDAMENTO DE CONSULTAS MÉDICAS

Modelagem de Software — APS 1

Trabalho acadêmico apresentado à disciplina de Modelagem de Software do curso de Bacharelado em Engenharia de Software da Universidade Federal de Goiás (UFG), como requisito parcial de avaliação referente à Atividade Prática Supervisionada (APS 1).

Professor Orientador: Prof. Me. Diogo Oliveira Santos

GOIÂNIA

2026

SUMÁRIO

1 Descrição da Aplicação **3**
1.1 Contexto e Objetivos 3
1.2 Problema de Negócio e Solução Proposta 3
1.3 Atores do Sistema 3
1.4 Módulos Funcionais e Regras de Negócio 4
1.5 Ciclo de Vida da Consulta 5
1.6 Quem Recebe Cada Notificação 6
1.7 Escopo e Premissas 6

2 Diagramas de Atividades **6**
2.1 DA-01 — Agendamento de Consulta 6

3 Diagramas de Sequência **8**
3.1 DS-01 — Agendamento de Consulta (Fluxo Completo) 8

4 Diagrama de Classes — Nível Macro **11**
4.1 DC — Diagrama de Classes (Nível Macro) 11

5 Considerações Finais **13**

REFERÊNCIAS **15**

A Especificação de Requisitos e Regras de Negócio **16**
A.1 Requisitos Funcionais (RF) 16
A.1.1 Módulo de Cadastro e Autenticação 16
A.1.2 Módulo de Consulta de Médicos e Especialidades 16
A.1.3 Módulo de Agendamento 17
A.1.4 Módulo de Agenda do Médico 17
A.1.5 Módulo de Confirmação (Secretário) 18
A.1.6 Módulo de Notificação 18
A.2 Regras de Negócio (RN) 18
A.2.1 Identificação e acesso 18
A.2.2 Cadastros e catálogo 18
A.2.3 Disponibilidade e horários 19
A.2.4 Ciclo de vida da consulta 19
A.2.5 Confirmação e notificação 20

B Código-fonte PlantUML dos Diagramas **21**
B.1 DA-01 — Agendamento de Consulta 21
B.2 DS-01 — Agendamento de Consulta (Fluxo Completo) 23
B.3 DC — Diagrama de Classes (Nível Macro) 26

1 DESCRIÇÃO DA APLICAÇÃO

1.1 Contexto e Objetivos

Este trabalho apresenta a modelagem de um sistema web para utilização em clínicas e consultórios, que permite que os usuários realizem agendamentos de consultas e, para a clínica, o gerenciamento da agenda de atendimento de diversos médicos e suas especialidades.

Temos como objetivo principal dar autonomia ao usuário para que ele consiga, a qualquer momento, realizar um agendamento na clínica desejada, selecionando a especialidade e o médico e definindo o horário, sem precisar fazer ligações ou enviar mensagens, agilizando assim o processo de agendamento, remarcação e cancelamento de consultas. Além disso, a equipe da clínica passa a ter um melhor gerenciamento da agenda de seus médicos especialistas, ganhando agilidade no processo e permitindo que os próprios médicos cadastrem seus horários, enquanto o administrador tem uma visão geral do funcionamento da clínica.

1.2 Problema de Negócio e Solução Proposta

Processos manuais em clínicas costumam gerar sobrecarga na recepção, marcações duplas no mesmo horário e alto índice de faltas pela falta de lembretes práticos. Além disso, controles em planilhas ou papel dificultam o remanejamento rápido de horários cancelados e não oferecem histórico confiável para auditoria.

O sistema resolve esses problemas ao centralizar as agendas em uma base única, aplicando validação em tempo real e bloqueio de horários para impedir conflitos entre pacientes que desejam um mesmo horário. Para reduzir as faltas, a aplicação envia confirmações e lembretes simultaneamente por e-mail e WhatsApp, os dois canais mais usados no dia a dia. Além disso, todas as ações críticas (agendamentos, confirmações, cancelamentos e remarcações com motivo obrigatório e alterações de grade) geram registros de auditoria com autor, data/hora, IP e navegador, apoiando a rastreabilidade e a conformidade com a LGPD. Ao armazenar o motivo de cada cancelamento ou remarcação, cria-se um histórico que ajuda a clínica a entender por que os horários são liberados.

1.3 Atores do Sistema

O sistema conta com cinco atores bem definidos:

1. **Paciente:** cadastra-se com CPF, e-mail e telefone celular; autentica-se com senha segura; pesquisa médicos por especialidade; consulta a grade de horários disponíveis; e gerencia suas consultas (agendamento, histórico, remarcação e cancelamento). Recebe notificações automáticas por e-mail e WhatsApp, inclusive um lembrete 24 horas antes da consulta.
2. **Médico:** profissional cadastrado com CRM, UF e especialidades ativas. Configura sua disponibilidade semanal (dias de atendimento, horário de expediente e duração das consultas); bloqueia horários pontuais por conveniência; visualiza sua agenda; confirma as próprias consultas; e registra o status do atendimento (iniciando, concluindo ou registrando

a ausência do paciente). Não agenda, cancela nem remarca consultas.

3. **Secretário:** profissional da recepção. Acompanha as agendas de todos os médicos, gerencia as consultas pendentes de confirmação (com contato proativo com pacientes quando necessário), registra confirmações ou recusas e pode agendar, remarcar ou cancelar consultas a pedido de pacientes presenciais ou por telefone.
4. **Administrador:** responsável pelos cadastros dentro do sistema. Cadastra e mantém médicos e secretários e gerencia o catálogo de especialidades (ativar e desativar). Não agenda nem altera consultas.
5. **Sistema de Notificação:** componente secundário que roda em segundo plano. Funciona a partir dos eventos gerados pelas operações da clínica e envia as mensagens em paralelo por e-mail e WhatsApp, com fila de retentativas automáticas caso ocorra instabilidade temporária na rede, garantindo que o agendamento nunca trave por falha de envio.

1.4 Módulos Funcionais e Regras de Negócio

O sistema é organizado em seis módulos complementares:

- **Módulo de Cadastro e Autenticação:** responsável pelo gerenciamento e pela autenticação de todos os usuários da clínica e pacientes. Exige e-mail único para todos e CPF único para pacientes. Médicos são cadastrados pelo par CRM/UF, dados únicos nacionalmente. Os perfis de acesso são exclusivos (`PACIENTE`, `MEDICO`, `SECRETARIO`, `ADMINISTRADOR`). Pacientes se cadastram sozinhos; médicos e secretários são cadastrados pelo administrador. As senhas são armazenadas sob hash criptográfico, com bloqueio temporário após sucessivas tentativas incorretas.
- **Módulo de Consulta de Médicos e Especialidades:** o administrador gerencia a lista de especialidades (apenas ativas aceitam novos agendamentos). Permite a busca pública de especialidades e a filtragem de médicos ativos vinculados a cada catálogo.
- **Módulo de Agenda do Médico:** organiza a grade do profissional em intervalos de **30 minutos por padrão**, configuráveis pelo médico em blocos de 30 minutos, sem sobreposição. Cada intervalo possui estado bem definido (`LIVRE`, `BLOQUEADO` ou `OCUPADO`). Os horários ficam disponíveis para reserva dentro do expediente do médico, respeitando uma antecedência mínima configurável e um limite máximo de até 90 dias no futuro. Permite ao médico aplicar bloqueios operacionais pontuais.
- **Módulo de Agendamento:** guia o paciente em quatro passos (especialidade, médico, data/horário e confirmação do resumo) e centraliza todo o ciclo da reserva. Ao confirmar, o sistema reserva o horário utilizando trava atômica (bloqueio exclusivo no banco de dados) para impedir marcações concorrentes no mesmo slot e grava a consulta como `AGENDADA`. Abrange também o histórico de consultas e as operações de **cancelamento e remarcação** (pelo paciente titular ou pelo secretário): ambas exigem antecedência mínima de 2 horas (bloqueando a ação no sistema caso falte menos de 2 h) e **motivo obrigatório**. No

cancelamento, o slot volta imediatamente a LIVRE. A remarcação mantém o mesmo médico, cancelando a consulta antiga, ocupando o novo horário e criando uma nova consulta AGENDADA com reversão automática em caso de falha.

- **Módulo de Confirmação (Secretário):** apoia a rotina da recepção através de uma visão centralizada das consultas pendentes de confirmação. Permite o contato proativo com os pacientes presenciais ou telefônicos e o registro de confirmações (transicionando a consulta para CONFIRMADA) ou recusas, gravando autor e data da confirmação. A confirmação é **opcional**, permitindo que o médico inicie o atendimento mesmo que a consulta permaneça com status AGENDADA.
- **Módulo de Notificação:** funciona em segundo plano, disparando avisos assíncronos e em paralelo por e-mail e WhatsApp nos eventos de agendamento, confirmação, cancelamento, remarcação e lembrete prévio de 24 horas. Cada canal possui registro próprio de entrega com fila de até 3 tentativas e backoff. Uma falha de entrega não cancela a consulta nem interrompe o fluxo do usuário. Os eventos críticos do sistema são registrados em log imutável de auditoria com rastreabilidade de autor, data/hora e IP.

1.5 Ciclo de Vida da Consulta

Quadro 1 – Ciclo de vida da consulta

De	Evento	Para	Quem pode
—	agendar	AGENDADA	Paciente, Secretário
AGENDADA	confirmar	CONFIRMADA	Secretário, Médico titular
AGENDADA, CONFIRMADA	iniciar	EM_ANDAMENTO	Médico titular
AGENDADA, CONFIRMADA	cancelar (≥ 2 h, com motivo)	CANCELADA	Paciente titular, Secretário
AGENDADA, CONFIRMADA	remarcar (≥ 2 h, com motivo)	REMARCADA (e nova AGENDADA)	Paciente titular, Secretário
AGENDADA, CONFIRMADA	horário passou sem início	NAO_REALIZADA	Sistema
AGENDADA, CONFIRMADA, EM_ANDAMENTO	registrar ausência	NAO_REALIZADA	Médico titular
EM_ANDAMENTO	encerrar	REALIZADA	Médico titular

Fonte: Elaborado pelos autores (2026).

CANCELADA, REMARCADA, REALIZADA e NAO_REALIZADA são estados finais de cada consulta.

1.6 Quem Recebe Cada Notificação

Cada evento gera uma notificação por destinatário e por canal (e-mail e WhatsApp).

Quadro 2 – Destinatários das notificações por evento

Evento	Paciente	Médico
Agendamento	sim	não
Confirmação	sim	sim
Cancelamento	sim	sim
Remarcação	sim	sim
Lembrete (24 h antes)	sim	não

Fonte: Elaborado pelos autores (2026).

1.7 Escopo e Premissas

- A aplicação é um **web app responsivo**; não há aplicativo nativo.
- Ficam **fora do escopo**: prontuário eletrônico, prescrições, pagamento, convênios, nota fiscal, teleconsulta e várias clínicas (multiclínica).
- Há um único contexto de clínica.
- A auditoria cobre eventos de negócio (criar, confirmar, cancelar, remarcar, bloquear horário e alterar cadastro), não leituras.

2 DIAGRAMAS DE ATIVIDADES

Fizemos um único diagrama de atividades, o **DA-01**, com o fluxo principal de agendamento e todos os caminhos alternativos e de erro. Como ele já cobre o que foi pedido, não criamos diagramas separados para cancelar, remarcar e confirmar consulta; esses processos estão explicados nas seções 1.4 e 1.5.

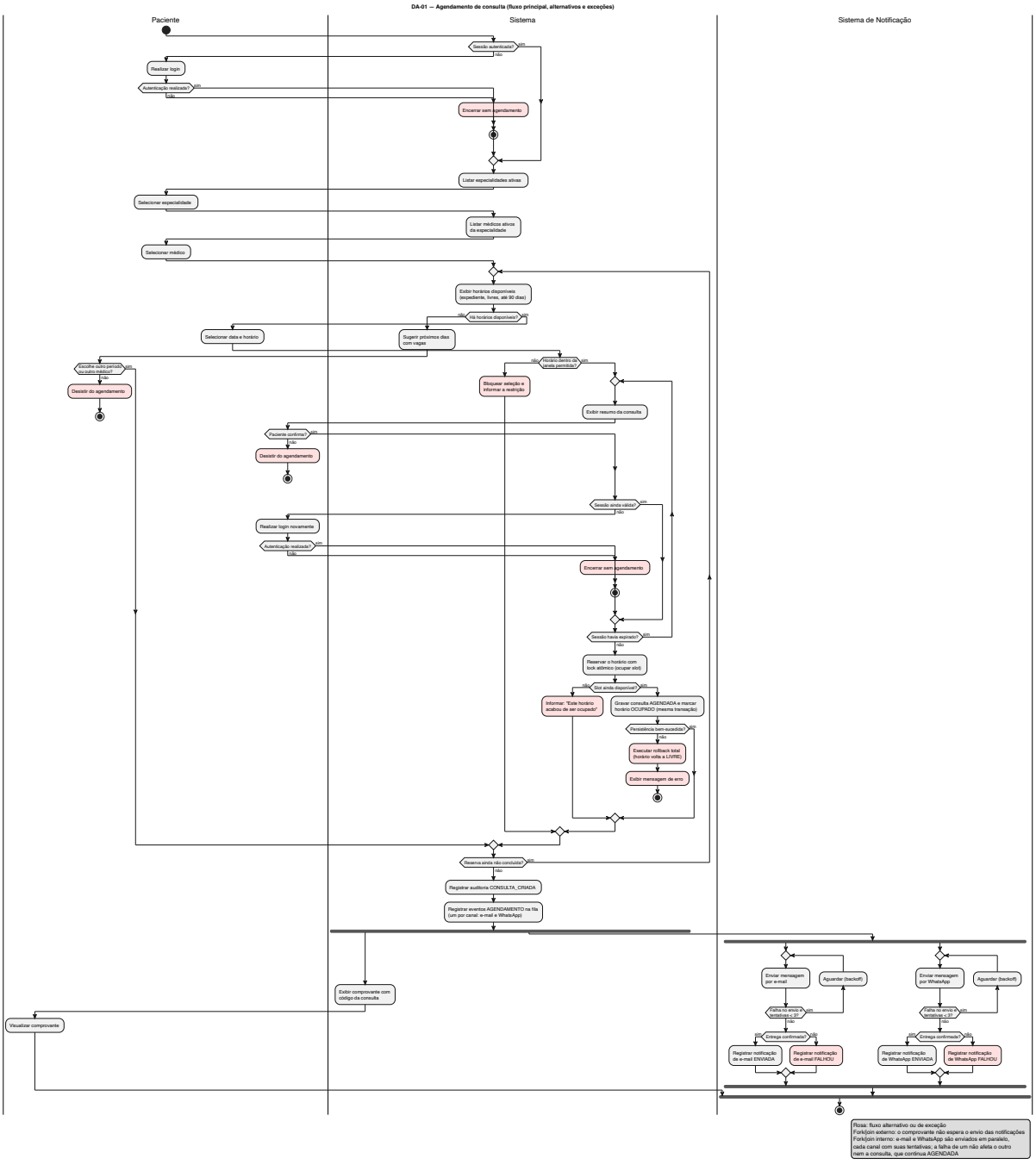
2.1 DA-01 — Agendamento de Consulta

Objetivo: mostrar o passo a passo do paciente para agendar uma consulta, do login até receber o comprovante, incluindo o que acontece quando algo dá errado.

Quem participa (raias): Paciente, Sistema e Sistema de Notificação.

Como funciona: o paciente faz login (se ainda não estiver logado), escolhe a especialidade, o médico e um horário livre, confere o resumo e confirma. O sistema reserva o horário de uma vez só, para dois pacientes não pegarem a mesma vaga, grava a consulta como `AGENDADA`, marca o horário como `OCUPADO` e registra a auditoria. Depois disso, o fluxo se divide em duas partes que andam ao mesmo tempo: o sistema mostra o comprovante ao paciente e o Sistema de Notificação envia as mensagens. O envio também se divide em dois, e-mail e WhatsApp, cada um com até 3 tentativas e com seu próprio resultado (`ENVIADA` ou `FALHOU`). No fim tudo se junta, mas o comprovante não fica esperando o envio.

Figura 1 – DA-01 — Diagrama de atividades: agendamento de consulta



Fonte: Elaborado pelos autores (2026).

Como ler o diagrama: as raias separam o que cada ator faz. Os laços (`repeat`) voltam para a escolha de horário quando não há vagas, quando o horário está fora da janela ou quando outro paciente pegou o horário, e voltam para o resumo depois de um novo login. Os blocos rosa são os caminhos alternativos e de erro. O *fork/join* mostra o que acontece ao mesmo tempo: o comprovante aparece enquanto as mensagens são enviadas, e se o envio falhar o agendamento continua valendo.

Onde cada fluxo alternativo (A) e de erro (E) aparece no DA-01:

Quadro 3 – Fluxos alternativos e de exceção do DA-01

Fluxo	Onde aparece no diagrama
Login inicial	No começo, em "Sessão autenticada?"; se o login falhar, o fluxo termina
A1 — sem horários	"Há horários disponíveis?" não → o sistema sugere outros dias → o paciente escolhe outro período ou médico (volta para a escolha de horário) ou desiste
A2 — horário tomado por outro paciente	"Slot ainda disponível?" não → aviso "Este horário acabou de ser ocupado" → volta para a escolha de horário
A3 — falha no envio da mensagem	No Sistema de Notificação: falha no envio → até 3 tentativas → `FALHOU`; a consulta continua `AGENDADA`
E1 — sessão expirou	"Sessão ainda válida?" não → novo login → volta para o resumo da consulta
E2 — falha ao gravar	"Persistência bem-sucedida?" não → desfaz tudo (o horário volta a `LIVRE`) → mensagem de erro
E3 — horário fora da janela	"Horário dentro da janela permitida?" não → bloqueia a seleção e explica a restrição
Desistências	"Paciente confirma?" não, ou "Escolhe outro período ou outro médico?" não

Fonte: Elaborado pelos autores (2026).

3 DIAGRAMAS DE SEQUÊNCIA

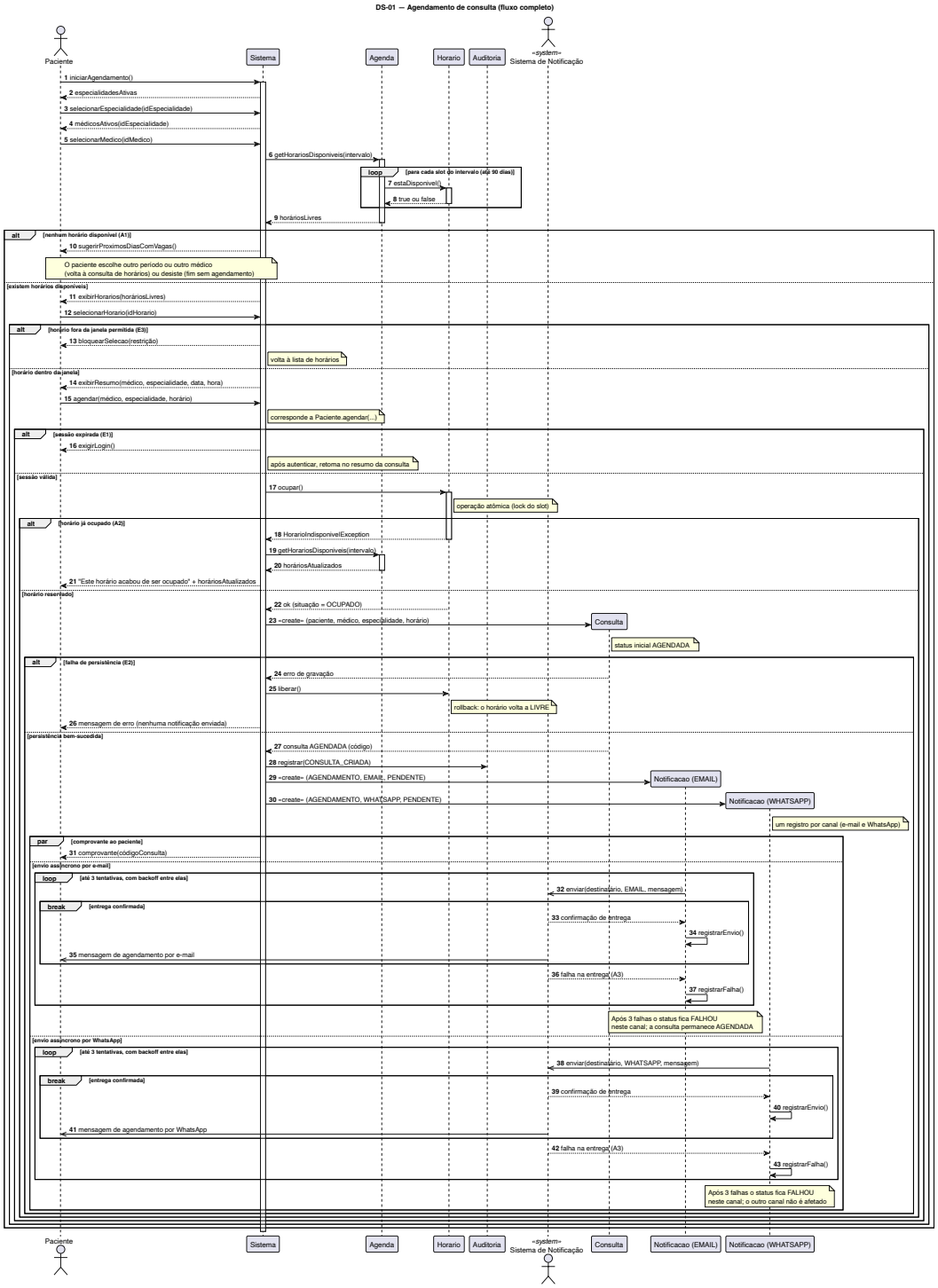
O **DS-01** cobre o mesmo cenário do DA-01 sob uma perspectiva diferente: enquanto o diagrama de atividades mapeia *o que* acontece, o diagrama de sequência revela *como* os participantes se comunicam para que isso aconteça. Ele ordena cronologicamente as mensagens trocadas entre o paciente, o sistema, os objetos de negócio (`Agenda`, `Horario`, `Consulta`, `Notificacao`) e o Sistema de Notificação, incluindo os retornos e todos os cenários alternativos e de exceção.

3.1 DS-01 — Agendamento de Consulta (Fluxo Completo)

Objetivo: documentar, mensagem a mensagem, o que acontece durante um agendamento. Os pontos centrais são a reserva atômica do horário — que previne conflitos de concorrência —, a criação da consulta e o rollback em caso de falha, e o envio assíncrono e paralelo das notificações por e-mail e WhatsApp.

Cenário representado: um paciente autenticado seleciona especialidade, médico e horário, e conclui o agendamento. O diagrama trata todas as situações excepcionais relevantes: ausência de horários disponíveis, seleção de horário fora da janela permitida, sessão expirada, tentativa de reservar um horário já ocupado por outro paciente, falha de gravação no banco e falha no envio da notificação.

Figura 2 – DS-01 — Diagrama de sequência: agendamento de consulta



Fonte: Elaborado pelos autores (2026).

Explicação dos pontos-chave:

- **Reserva atômica (mensagens 17–18):** quando o paciente confirma o horário, o sistema chama `Horario.ocupar()` em uma única operação indivisível. Se outro paciente chegou primeiro, o método lança `HorarioIndisponivelException` e o fluxo segue para A2 — nunca existe uma janela de tempo entre "verificar disponibilidade" e "efetivar a reserva"

que permita duas confirmações simultâneas do mesmo slot.

- **Rollback em caso de falha (mensagem 25):** se a criação da `Consulta` falha por problema de persistência (E2), o sistema desfaz imediatamente a reserva chamando `Horario.liberar()`, devolvendo o slot ao estado `LIVRE`. Nenhuma notificação é disparada nesse caso, garantindo consistência entre os dados e o que o paciente recebe.
- **Criação de objetos em tempo de execução (mensagens 23, 29 e 30):** `Consulta` e os dois objetos `Notificacao` — um por canal — só existem a partir do momento em que são criados no fluxo (setas `<<create>>`). Isso torna explícito que a criação é parte do processo de agendamento, e não algo pré-existente. A mensagem 15, `agendar(...)`, corresponde à operação `Paciente.agendar(medico, especialidade, horario)` do diagrama de classes.
- **Paralelismo no envio (mensagem 31 em diante):** assim que a consulta é gravada com sucesso, o bloco `par` divide o fluxo em três ramos simultâneos: entrega do comprovante ao paciente, envio por e-mail e envio por WhatsApp. Esse comportamento espelha os *forks* do DA-01. As setas com ponta aberta (`->>`) indicam que as chamadas ao Sistema de Notificação são assíncronas — o sistema não espera a confirmação de entrega para liberar o paciente.
- **Retentativas independentes por canal (mensagens 32–43):** cada canal possui seu próprio laço de até 3 tentativas com *backoff* entre elas. Quando uma entrega é confirmada, o bloco `break` encerra o laço daquele canal. Após 3 falhas consecutivas, o status do objeto `Notificacao` passa para `FALHOU` — mas isso afeta apenas o canal em questão; o outro canal continua normalmente e o status da consulta permanece `AGENDADA`.
- **Papel do participante `Sistema`:** representa a camada de aplicação, que orquestra todo o fluxo. As mensagens que chegam a ele (`iniciarAgendamento`, `selecionarMedico`, `selecionarHorario` e outras) são operações do controlador da aplicação, e não métodos diretos das classes de domínio. Essa separação é intencional: ela mantém o diagrama focado na lógica de negócio e na interação entre os objetos, sem misturar detalhes de telas ou de interface com as classes do sistema.

A tabela abaixo mapeia cada fluxo alternativo (A) e de exceção (E) às mensagens do diagrama onde ele é tratado:

Quadro 4 – Correspondência dos fluxos com as mensagens do DS-01

Fluxo	Mensagens
A1 — sem horários	10
E3 — fora da janela	13
E1 — sessão expirada	16
A2 — horário ocupado	17 a 21
E2 — falha de persistência	24 a 26
A3 — falha na notificação	36 e 37 (e-mail) e 42 e 43 (WhatsApp), em laços com até 3 tentativas

Fonte: Elaborado pelos autores (2026).

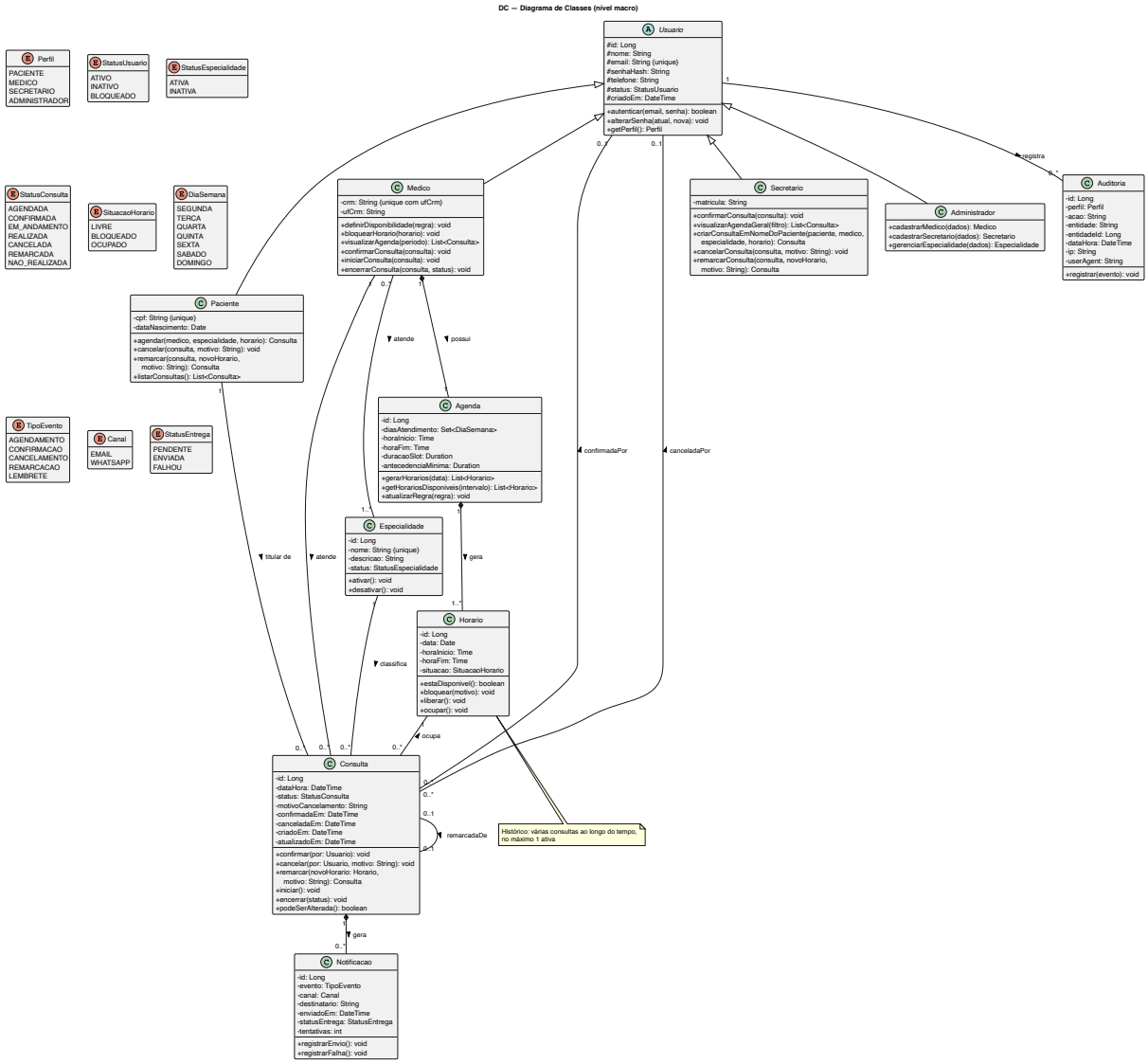
4 DIAGRAMA DE CLASSES — NÍVEL MACRO

O diagrama de classes estrutura a visão estática do domínio com 11 classes e 9 enumerações, mapeando os relacionamentos essenciais, a visibilidade dos membros e as regras de multiplicidade para guiar a implementação.

4.1 DC — Diagrama de Classes (Nível Macro)

Objetivo: apresentar a estrutura orientada a objetos do sistema: classes principais, atributos tipados, métodos essenciais, enumerações e relacionamentos (herança, associação e composição), com multiplicidades e visibilidade (+ público, − privado, # protegido).

Figura 3 – Diagrama de classes (nível macro)



Fonte: Elaborado pelos autores (2026).

Explicação dos Elementos e Notação UML

Na UML, o símbolo + indica visibilidade pública, − representa privada e # indica protegida. Os atributos da classe `Usuario` usam # (protegidos) para permitir que as subclasses os herdem e acessem diretamente; para as demais classes, o padrão é manter atributos privados e métodos públicos. Nos relacionamentos, o losango preenchido marca a composição, o triângulo vazio indica a herança e a linha simples mostra a associação.

Estrutura de Perfis (Herança)

A classe abstrata `Usuario` centraliza as informações básicas de identificação e autenticação (`id`, `nome`, `email`, `senhaHash`, `telefone`, `status` e `criadoEm`), além de conter os métodos de segurança. A partir dela, o sistema se divide em quatro perfis estritos de acesso:

- **Paciente:** adiciona o atributo `cpf` ao cadastro.
- **Medico:** exige as credenciais de `crm` e `ufCrm`.
- **Secretario:** inclui o atributo `matricula`.
- **Administrador:** agrupa as operações exclusivas de gestão e cadastro da clínica.

Dependência de Existência (Composição)

Neste tipo de ligação estrutural, o objeto-todo governa o ciclo de vida das suas partes:

- **Medico – Agenda (1:1):** todo médico possui uma agenda única, que perde o sentido e deixa de existir caso o profissional seja desvinculado.
- **Agenda – Horario (1:1..*):** a agenda fragmenta o expediente em slots de atendimento; sem a agenda matriz, esses horários não existem de forma autônoma.
- **Consulta – Notificacao (1:0..*):** as mensagens pertencem à consulta que as originou, mantendo o registro de envios atrelado ao evento clínico.

Vínculos Operacionais (Associações)

- **Medico – Especialidade (0..*:1..*):** um médico atende a uma ou múltiplas especialidades, e uma especialidade pode ter vários médicos vinculados.
- **Horario – Consulta (1:0..*):** um mesmo horário retém o histórico de todas as consultas que passaram por ele ao longo do tempo (como uma consulta cancelada que cedeu espaço para outra), mas admite no máximo uma ativa por vez.
- **Paciente – Consulta (1:0..*):** cada consulta pertence obrigatoriamente a exatamente um paciente titular, enquanto o paciente acumula o histórico de seus agendamentos.
- **Medico – Consulta (1:0..*):** cada consulta é conduzida por exatamente um médico responsável, enquanto o médico atende a diversas consultas na clínica.
- **Especialidade – Consulta (1:0..*):** fixa qual especialidade foi selecionada no momento da marcação, já que o médico pode atuar em mais de uma.

- **Consulta – Consulta (0..1:0..1, `remarcadaDe`):** associação reflexiva na qual a nova consulta aponta para a consulta anterior cancelada, sendo opcional (`0..1`) pois agendamentos comuns não derivam de outra.
- **Consulta – Usuario (0..*:0..1, `confirmadaPor` e `canceladaPor`):** identifica qual operador executou as ações. Cruzando isso com `confirmadaEm`, `canceladaEm` e `motivoCancelamento`, obtém-se a rastreabilidade completa do ciclo de vida da consulta.
- **Usuario – Auditoria (1:0..*):** associação simples (e não composição) para assegurar que os logs de auditoria sobrevivam de forma imutável e independente da conta do usuário.

Domínios e Regras (Enumerações)

- `Perfil`: trava as contas em `PACIENTE`, `MEDICO`, `SECRETARIO` ou `ADMINISTRADOR`.
- `StatusUsuario`: restringe a situação cadastral a `ATIVO`, `INATIVO` ou `BLOQUEADO`.
- `StatusEspecialidade`: chaveia a disponibilidade no catálogo entre `ATIVA` ou `INATIVA`.
- `StatusConsulta`: mapeia o ciclo de atendimento em `AGENDADA`, `CONFIRMADA`, `EM_ANDAMENTO`, `REALIZADA`, `CANCELADA`, `REMARCADA` ou `NAO_REALIZADA`.
- `SituacaoHorario`: controla o estado de cada vaga da grade como `LIVRE`, `BLOQUEADO` ou `OCUPADO`.
- `DiaSemana`: parametriza os dias de atendimento configurados na agenda do médico.
- `TipoEvento`: filtra os gatilhos de comunicação em `AGENDAMENTO`, `CONFIRMACAO`, `CANCELAMENTO`, `REMARCACAO` ou `LEMBRETE`.
- `Canal`: limita os disparos de notificação a `EMAIL` ou `WHATSAPP`.
- `StatusEntrega`: sinaliza o resultado do envio via `PENDENTE`, `ENVIADA` ou `FALHOU`.

5 CONSIDERAÇÕES FINAIS

A modelagem desenvolvida mostra como a UML organiza e comunica a estrutura e o comportamento de um sistema antes da implementação, conectando as regras de negócio aos modelos de software.

O **diagrama de atividades** (DA-01) descreve o processo de agendamento de ponta a ponta. Com raias para o paciente, o sistema e o Sistema de Notificação, ele cobre o fluxo principal e todos os fluxos alternativos e de exceção (sem vagas, horário ocupado, horário fora da janela, sessão expirada, falha de gravação e falha de notificação) e usa *fork/join* (divisão e junção de fluxos paralelos) em dois níveis: no primeiro, o comprovante é gerado em paralelo ao envio das notificações; no segundo, dentro do envio, o e-mail e o WhatsApp saem em paralelo, cada um com suas próprias tentativas.

O **diagrama de sequência** (DS-01) mostra, no mesmo cenário, a troca de mensagens ao longo do tempo. Seus destaques são a reserva atômica do horário, ou seja, de uma só vez, sem risco de dois pacientes ficarem com o mesmo horário; o *rollback* (desfazer a operação) em caso de falha; a criação da consulta e o envio assíncrono e paralelo por e-mail e WhatsApp, com até 3 tentativas em cada canal.

O **diagrama de classes** estrutura a visão estática do domínio com 11 classes e 9 enumerações, relacionamentos de herança, composição e associação com multiplicidades, e indica a visibilidade dos atributos e métodos.

Todos os diagramas foram revisados para garantir que estivessem coerentes entre si. Dessa revisão resultaram decisões que tornaram o modelo mais fiel às regras de negócio, como a multiplicidade entre `Horario` e `Consulta`, o registro de quem confirmou e quem cancelou, a reserva atômica do horário e a separação entre o comprovante e o envio da notificação.

Limitações e próximos passos: a remarcação, a confirmação pelo secretário e o cancelamento estão descritos em texto (seções 1.4 e 1.5), mas não têm diagrama próprio, porque o DA-01 e o DS-01 já cobrem o que foi pedido. Se houver evolução do trabalho, esses cenários são os candidatos naturais a novos diagramas (por exemplo, o cancelamento com bloqueio da janela de 2 horas).

REFERÊNCIAS

FOWLER, Martin. **UML essencial**: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

OBJECT MANAGEMENT GROUP (OMG). **OMG Unified Modeling Language (OMG UML)**: Superstructure, Version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/. Acesso em: 8 out. 2026.

PLANTUML. **PlantUML Language Reference Guide**. 2024. Disponível em: https://plantuml.com/guide. Acesso em: 8 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

APÊNDICE A — ESPECIFICAÇÃO DE REQUISITOS E REGRAS DE NEGÓCIO

As decisões de modelagem apresentadas neste trabalho foram fundamentadas em um levantamento prévio de requisitos funcionais e regras de negócio, consolidados abaixo para fins de consulta e rastreabilidade.

A.1 Requisitos Funcionais (RF)

A.1.1 Módulo de Cadastro e Autenticação

ID	Requisito	Descrição	Regras relacionadas
RF-01	Cadastrar paciente	O sistema deve permitir o cadastro de paciente com nome, CPF, data de nascimento, e-mail, telefone e senha.	RN-01, RN-02, RN-03
RF-02	Autenticar usuário	O sistema deve autenticar paciente, médico, secretário e administrador por e-mail + senha, diferenciando perfis.	RN-04, RN-06
RF-03	Recuperar senha	O sistema deve permitir redefinição de senha por link enviado ao e-mail cadastrado.	RN-05
RF-04	Gerenciar cadastro de médico e secretário	O **Administrador** deve poder cadastrar médico (nome, CRM, UF, especialidade(s) e contato) e secretário (nome, matrícula e contato).	RN-02, RN-07, RN-08
RF-05	Gerenciar especialidades	O **Administrador** deve poder manter o catálogo de especialidades (nome, descrição, status ATIVA/INATIVA).	RN-09

Fonte: Elaborado pelos autores (2026).

A.1.2 Módulo de Consulta de Médicos e Especialidades

ID	Requisito	Descrição	Regras relacionadas
RF-06	Listar especialidades	O sistema deve exibir todas as especialidades ativas para busca.	RN-09
RF-07	Listar médicos por especialidade	Dada uma especialidade, o sistema deve listar os médicos ativos que a atendem.	RN-07, RN-10
RF-08	Exibir horários disponíveis	Para cada médico, o sistema deve exibir apenas os horários **livres** dentro da sua disponibilidade, considerando o horário atual.	RN-11, RN-12, RN-13

Fonte: Elaborado pelos autores (2026).

A.1.3 Módulo de Agendamento

ID	Requisito	Descrição	Regras relacionadas
RF-09	Agendar consulta	O paciente deve poder agendar consulta selecionando especialidade → médico → data → horário livre, com confirmação final antes do envio.	RN-10, RN-11, RN-13, RN-16, RN-18
RF-10	Confirmar agendamento	Após a reserva bem-sucedida, o sistema deve exibir resumo da consulta e disparar notificação de agendamento.	RN-25, RN-27
RF-11	Visualizar minhas consultas	O paciente deve poder listar suas consultas futuras e passadas com respectivo status.	RN-19
RF-12	Cancelar consulta	O paciente (ou secretário) deve poder cancelar uma consulta futura, informando o motivo (obrigatório), liberando o horário.	RN-20, RN-21
RF-13	Remarcar consulta	O paciente titular (ou o secretário) deve poder remarcar uma consulta futura **com o mesmo médico**, informando o motivo (obrigatório) e validando o novo horário como uma nova reserva.	RN-13, RN-17, RN-20, RN-21
RF-22	Agir em nome do paciente	O secretário deve poder agendar, remarcar e cancelar consultas em nome de um paciente cadastrado (atendimento presencial/telefônico), com as mesmas validações de RF-09, RF-12 e RF-13.	RN-10, RN-11, RN-13, RN-16, RN-18, RN-20, RN-23

Fonte: Elaborado pelos autores (2026).

A.1.4 Módulo de Agenda do Médico

ID	Requisito	Descrição	Regras relacionadas
RF-14	Definir disponibilidade	O médico deve poder definir dias da semana e faixas horárias de atendimento, com antecedência mínima configurável.	RN-11, RN-12, RN-13, RN-15
RF-15	Bloquear horário	O médico deve poder bloquear um horário específico já disponível (sem consulta), tornando-o indisponível. O secretário pode registrar bloqueios operacionais justificados.	RN-14, RN-15
RF-16	Visualizar agenda	O médico e o secretário devem poder visualizar a agenda em visão diária/semanal, com status de cada consulta.	RN-19
RF-17	Atender e encerrar consulta	O médico deve poder iniciar a consulta, marcá-la como **realizada** ao final do atendimento ou registrar a **ausência** do paciente (`NAO_REALIZADA`).	RN-22, RN-24

Fonte: Elaborado pelos autores (2026).

A.1.5 Módulo de Confirmação (Secretário)

ID	Requisito	Descrição	Regras relacionadas
RF-18	Confirmar consulta	O secretário (ou o médico titular) deve poder confirmar uma consulta com status *agendada*, alterando-a para *confirmada* e registrando quem confirmou e quando.	RN-24, RN-25, RN-26
RF-19	Visualizar pendências	O sistema deve listar as consultas pendentes de confirmação, filtráveis por médico, data e status.	RN-26

Fonte: Elaborado pelos autores (2026).

A.1.6 Módulo de Notificação

ID	Requisito	Descrição	Regras relacionadas
RF-20	Notificar eventos	O sistema deve disparar notificações nos eventos: agendada, confirmada, cancelada, remarcada, lembrete pré-consulta.	RN-25, RN-27, RN-28
RF-21	Registrar notificações	Cada notificação deve ser persistida com destinatário, evento, canal, data/hora e status de entrega.	RN-27

Fonte: Elaborado pelos autores (2026).

A.2 Regras de Negócio (RN)

A.2.1 Identificação e acesso

ID	Regra
RN-01	O CPF deve ter 11 dígitos, ser válido (módulo 11) e **único** no cadastro de pacientes.
RN-02	O e-mail deve ser válido e **único** por usuário no sistema.
RN-03	A senha deve ter no mínimo 8 caracteres, contendo ao menos 1 letra e 1 número.
RN-04	O sistema autentica por e-mail + senha e deriva o perfil (PACIENTE, MEDICO, SECRETARIO, ADMINISTRADOR) do cadastro.
RN-05	Links de redefinição de senha expiram em 30 minutos e são de uso único.
RN-06	Após 5 falhas consecutivas de login, a conta é bloqueada por 15 minutos.

Fonte: Elaborado pelos autores (2026).

A.2.2 Cadastros e catálogo

ID	Regra
RN-07	Todo médico deve ter **ao menos uma** especialidade ativa para aparecer nas buscas.
RN-08	O par (CRM, UF) do médico é **único** no sistema.
RN-09	Só são exibidas especialidades com `status = ATIVA`.
RN-10	Paciente, médico, secretário e administrador só são exibidos com `status = ATIVO`; cadastro inativo não pode gerar novas consultas.

Fonte: Elaborado pelos autores (2026).

A.2.3 Disponibilidade e horários

ID	Regra
RN-11	Um horário só é considerado **disponível** se: pertencer à disponibilidade do médico, estar dentro do expediente configurado, não estiver bloqueado e **não houver consulta ativa** (AGENDADA, CONFIRMADA ou EM_ANDAMENTO) naquele slot.
RN-12	Cada consulta ocupa **um slot** de **30 minutos por padrão** (configurável pelo médico em múltiplos de 30), sem sobreposição entre consultas do mesmo médico.
RN-13	Só é permitido agendar a partir de `agora + antecedênciaMinima` da agenda do médico (**padrão 30 min**) e com no máximo **90 dias** de antecedência.
RN-14	Bloqueios operacionais de horário feitos pelo secretário exigem justificativa e ficam registrados em auditoria.
RN-15	Alterações de disponibilidade ou bloqueios que atinjam consultas já existentes **não** as cancelam automaticamente; o sistema deve sinalizar conflito e exigir tratamento manual.

Fonte: Elaborado pelos autores (2026).

A.2.4 Ciclo de vida da consulta

ID	Regra
RN-16	Toda consulta nasce com status `AGENDADA` e está sempre associada a **1 paciente, 1 médico, 1 especialidade e 1 slot de horário**.
RN-17	A **remarcação** é modelada como: cancelamento da consulta original (status `REMARCADA`, com vínculo para a nova) + criação de nova consulta. A remarcação mantém o **mesmo médico**; trocar de médico exige cancelar e agendar novamente.
RN-18	Apenas uma consulta pode ocupar o mesmo (médico, data, horário) com status ativo; consultas canceladas ou remarcadas permanecem como histórico, vinculadas ao mesmo `Horario`.
RN-19	Consultas `AGENDADA` ou `CONFIRMADA` cujo slot terminou sem terem sido iniciadas transicionam automaticamente para `NAO_REALIZADA` (executado pelo Sistema/agendador).
RN-20	Cancelamento/remarcação só é permitido **até 2 horas antes** do horário da consulta.
RN-21	Cancelamentos e remarcações exigem **confirmação explícita** do usuário, registram autor (`canceladaPor`) e data (`canceladaEm`), e exigem **motivo obrigatório** (`motivoCancelamento`).
RN-22	Somente o médico titular pode iniciar, encerrar (`REALIZADA`) ou registrar ausência (`NAO_REALIZADA`) de uma consulta. `REALIZADA` só é alcançado a partir de `EM_ANDAMENTO`.
RN-23	Consulta criada, remarcada ou cancelada pelo secretário em nome do paciente registra o secretário como autor na auditoria e notifica o paciente conforme RN-25.
RN-24	A confirmação (`CONFIRMADA`) é **opcional**: o médico pode iniciar uma consulta `AGENDADA` ou `CONFIRMADA`.

Fonte: Elaborado pelos autores (2026).

A.2.5 Confirmação e notificação

ID	Regra
RN-25	Cada evento de negócio (`AGENDAMENTO`, `CONFIRMACAO`, `CANCELAMENTO`, `REMARCACAO`) dispara uma notificação por destinatário **e por canal** (e-mail e WhatsApp). Mudanças para `EM_ANDAMENTO`, `REALIZADA` e `NAO_REALIZADA` não geram notificação.
RN-26	Somente **secretário** ou o **médico titular** pode confirmar consulta; a confirmação registra o autor (`confirmadaPor`) e o timestamp (`confirmadaEm`).
RN-27	Falha no envio de notificação **não** desfaz a transação principal; a notificação entra em fila de nova tentativa (até 3 tentativas com backoff). Cada canal tem o seu registro de `Notificacao` e as suas tentativas.
RN-28	O lembrete é disparado **24 h antes** do horário da consulta, apenas para status `CONFIRMADA` ou `AGENDADA`.

Fonte: Elaborado pelos autores (2026).

APÊNDICE B — CÓDIGO-FONTE PLANTUML DOS DIAGRAMAS

Os diagramas apresentados neste trabalho foram elaborados em PlantUML. Os códigos-fonte que geraram os modelos são reproduzidos a seguir para fins de verificação e reprodutibilidade.

B.1 DA-01 — Agendamento de Consulta

```
@startuml DA01_Agendamento
title DA-01 -- Agendamento de consulta (fluxo principal, alternativos e exceções)

|Paciente|
start
|Sistema|
if (Sessão autenticada?) then (não)
  |Paciente|
  :Realizar login;
  if (Autenticação realizada?) then (não)
    |Sistema|
    #FFE0E0:Encerrar sem agendamento;
    stop
  else (sim)
  endif
else (sim)
endif

|Sistema|
:Listar especialidades ativas;
|Paciente|
:Selecionar especialidade;
|Sistema|
:Listar médicos ativos\nda especialidade;
|Paciente|
:Selecionar médico;

repeat
  |Sistema|
  :Exibir horários disponíveis\n(expediente, livres, até 90 dias);
  if (Há horários disponíveis?) then (não)
    :Sugerir próximos dias\ncom vagas;
    |Paciente|
    if (Escolhe outro período\nou outro médico?) then (não)
      #FFE0E0:Desistir do agendamento;
      stop
    else (sim)
    endif
  else (sim)
```

```
    |Paciente|
    :Selecionar data e horário;
    |Sistema|
    if (Horário dentro da\njanela permitida?) then (não)
      #FFE0E0:Bloquear seleção e\ninformar a restrição;
    else (sim)
      repeat
        |Sistema|
        :Exibir resumo da consulta;
        |Paciente|
        if (Paciente confirma?) then (não)
          #FFE0E0:Desistir do agendamento;
          stop
        else (sim)
        endif
        |Sistema|
        if (Sessão ainda válida?) then (não)
          |Paciente|
          :Realizar login novamente;
          if (Autenticação realizada?) then (não)
            |Sistema|
            #FFE0E0:Encerrar sem agendamento;
            stop
          else (sim)
          endif
        else (sim)
        endif
      repeat while (Sessão havia expirado?) is (sim) not (não)
      |Sistema|
      :Reservar o horário com\nlock atômico (ocupar slot);
      if (Slot ainda disponível?) then (não)
        #FFE0E0:Informar: "Este horário\nacabou de ser ocupado";
      else (sim)
        :Gravar consulta AGENDADA e marcar\nhorário OCUPADO (mesma transação);
        if (Persistência bem-sucedida?) then (não)
          #FFE0E0:Executar rollback total\n(horário volta a LIVRE);
          #FFE0E0:Exibir mensagem de erro;
          stop
        else (sim)
        endif
      endif
    endif
  endif
repeat while (Reserva ainda não concluída?) is (sim) not (não)

|Sistema|
:Registrar auditoria CONSULTA_CRIADA;
```

```
:Registrar eventos AGENDAMENTO na fila\n(um por canal: e-mail e WhatsApp);
fork
  :Exibir comprovante com\ncódigo da consulta;
  |Paciente|
  :Visualizar comprovante;
fork again
  |Sistema de Notificação|
  fork
    repeat
      :Enviar mensagem\npor e-mail;
    backward:Aguardar (backoff);
    repeat while (Falha no envio e\ntentativas < 3?) is (sim) not (não)
    if (Entrega confirmada?) then (sim)
      :Registrar notificação\nde e-mail ENVIADA;
    else (não)
      #FFE0E0:Registrar notificação\nde e-mail FALHOU;
    endif
  fork again
    repeat
      :Enviar mensagem\npor WhatsApp;
    backward:Aguardar (backoff);
    repeat while (Falha no envio e\ntentativas < 3?) is (sim) not (não)
    if (Entrega confirmada?) then (sim)
      :Registrar notificação\nde WhatsApp ENVIADA;
    else (não)
      #FFE0E0:Registrar notificação\nde WhatsApp FALHOU;
    endif
  end fork
end fork
stop

legend right
  Rosa: fluxo alternativo ou de exceção
  Fork/join externo: o comprovante não espera o envio das notificações
  Fork/join interno: e-mail e WhatsApp são enviados em paralelo,
  cada canal com suas tentativas; a falha de um não afeta o outro
  nem a consulta, que continua AGENDADA
endlegend
@enduml
```

Fonte: Elaborado pelos autores (2026).

B.2 DS-01 — Agendamento de Consulta (Fluxo Completo)

```
@startuml DS01_Agendamento
title DS-01 -- Agendamento de consulta (fluxo completo)
autonumber
actor Paciente
```

```
participant "Sistema" as S
participant "Agenda" as AG
participant "Horario" as H
participant "Auditoria" as AUD
actor "Sistema de Notificação" as SN <<system>>

Paciente -> S : iniciarAgendamento()
activate S
S --> Paciente : especialidadesAtivas

Paciente -> S : selecionarEspecialidade(idEspecialidade)
S --> Paciente : médicosAtivos(idEspecialidade)

Paciente -> S : selecionarMedico(idMedico)
S -> AG : getHorariosDisponiveis(intervalo)
activate AG
loop para cada slot do intervalo (até 90 dias)
  AG -> H : estaDisponivel()
  activate H
  H --> AG : true ou false
  deactivate H
end
AG --> S : horáriosLivres
deactivate AG

alt nenhum horário disponível (A1)
  S --> Paciente : sugerirProximosDiasComVagas()
  note over Paciente, S
    O paciente escolhe outro período ou outro médico
    (volta à consulta de horários) ou desiste (fim sem agendamento)
  end note
else existem horários disponíveis
  S --> Paciente : exibirHorarios(horáriosLivres)
  Paciente -> S : selecionarHorario(idHorario)

  alt horário fora da janela permitida (E3)
    S --> Paciente : bloquearSelecao(restrição)
    note right of S : volta à lista de horários
  else horário dentro da janela
    S --> Paciente : exibirResumo(médico, especialidade, data, hora)
    Paciente -> S : agendar(médico, especialidade, horário)
    note right of S : corresponde a Paciente.agendar(...)

    alt sessão expirada (E1)
      S --> Paciente : exigirLogin()
      note right of S : após autenticar, retoma no resumo da consulta
    else sessão válida
```

```
52        S -> H : ocupar()
53        note right of H : operação atômica (lock do slot)
54        activate H
55
56        alt horário já ocupado (A2)
57          H --> S : HorarioIndisponivelException
58          deactivate H
59          S -> AG : getHorariosDisponiveis(intervalo)
60          activate AG
61          AG --> S : horáriosAtualizados
62          deactivate AG
63          S --> Paciente : "Este horário acabou de ser ocupado" +
    horáriosAtualizados
64        else horário reservado
65          H --> S : ok (situação = OCUPADO)
66          deactivate H
67
68          create Consulta
69          S -> Consulta : <<create>> (paciente, médico, especialidade, horário)
70          note right of Consulta : status inicial AGENDADA
71
72          alt falha de persistência (E2)
73            Consulta --> S : erro de gravação
74            S -> H : liberar()
75            note right of H : rollback: o horário volta a LIVRE
76            S --> Paciente : mensagem de erro (nenhuma notificação enviada)
77          else persistência bem-sucedida
78            Consulta --> S : consulta AGENDADA (código)
79            S -> AUD : registrar(CONSULTA_CRIADA)
80            create participant "Notificacao (EMAIL)" as NotifEmail
81            S -> NotifEmail : <<create>> (AGENDAMENTO, EMAIL, PENDENTE)
82            create participant "Notificacao (WHATSAPP)" as NotifWhats
83            S -> NotifWhats : <<create>> (AGENDAMENTO, WHATSAPP, PENDENTE)
84            note right of NotifWhats : um registro por canal (e-mail e WhatsApp)
85
86            par comprovante ao paciente
87              S --> Paciente : comprovante(códigoConsulta)
88            else envio assíncrono por e-mail
89              loop até 3 tentativas, com backoff entre elas
90                NotifEmail ->> SN : enviar(destinatário, EMAIL, mensagem)
91                break entrega confirmada
92                  SN --> NotifEmail : confirmação de entrega
93                  NotifEmail -> NotifEmail : registrarEnvio()
94                  SN ->> Paciente : mensagem de agendamento por e-mail
95                end
96                SN --> NotifEmail : falha na entrega (A3)
97                NotifEmail -> NotifEmail : registrarFalha()
```

```
            end
            note over NotifEmail
              Após 3 falhas o status fica FALHOU
              neste canal; a consulta permanece AGENDADA
            end note
          else envio assíncrono por WhatsApp
            loop até 3 tentativas, com backoff entre elas
              NotifWhats ->> SN : enviar(destinatário, WHATSAPP, mensagem)
              break entrega confirmada
                SN --> NotifWhats : confirmação de entrega
                NotifWhats -> NotifWhats : registrarEnvio()
                SN ->> Paciente : mensagem de agendamento por WhatsApp
              end
              SN --> NotifWhats : falha na entrega (A3)
              NotifWhats -> NotifWhats : registrarFalha()
            end
            note over NotifWhats
              Após 3 falhas o status fica FALHOU
              neste canal; o outro canal não é afetado
            end note
          end
        end
      end
    end
  end
end
deactivate S
@enduml
```

Fonte: Elaborado pelos autores (2026).

B.3 DC — Diagrama de Classes (Nível Macro)

```
@startuml DC_Classes
title DC -- Diagrama de Classes (nível macro)
skinparam classAttributeIconSize 0
hide empty members

' ===== ENUMERAÇÕES =====

enum Perfil {
  PACIENTE
  MEDICO
  SECRETARIO
  ADMINISTRADOR
}

enum StatusUsuario {
```

```
  ATIVO
  INATIVO
  BLOQUEADO
}

enum StatusEspecialidade {
  ATIVA
  INATIVA
}

enum StatusConsulta {
  AGENDADA
  CONFIRMADA
  EM_ANDAMENTO
  REALIZADA
  CANCELADA
  REMARCADA
  NAO_REALIZADA
}

enum SituacaoHorario {
  LIVRE
  BLOQUEADO
  OCUPADO
}

enum DiaSemana {
  SEGUNDA
  TERCA
  QUARTA
  QUINTA
  SEXTA
  SABADO
  DOMINGO
}

enum TipoEvento {
  AGENDAMENTO
  CONFIRMACAO
  CANCELAMENTO
  REMARCACAO
  LEMBRETE
}

enum Canal {
  EMAIL
  WHATSAPP
```

```
}

enum StatusEntrega {
  PENDENTE
  ENVIADA
  FALHOU
}

' ===== CLASSES =====

abstract class Usuario {
  # id: Long
  # nome: String
  # email: String {unique}
  # senhaHash: String
  # telefone: String
  # status: StatusUsuario
  # criadoEm: DateTime
  + autenticar(email, senha): boolean
  + alterarSenha(atual, nova): void
  + getPerfil(): Perfil
}

class Paciente {
  - cpf: String {unique}
  - dataNascimento: Date
  + agendar(medico, especialidade, horario): Consulta
  + cancelar(consulta, motivo: String): void
  + remarcar(consulta, novoHorario,\n    motivo: String): Consulta
  + listarConsultas(): List<Consulta>
}

class Medico {
  - crm: String {unique com ufCrm}
  - ufCrm: String
  + definirDisponibilidade(regra): void
  + bloquearHorario(horario): void
  + visualizarAgenda(periodo): List<Consulta>
  + confirmarConsulta(consulta): void
  + iniciarConsulta(consulta): void
  + encerrarConsulta(consulta, status): void
}

class Secretario {
  - matricula: String
  + confirmarConsulta(consulta): void
  + visualizarAgendaGeral(filtro): List<Consulta>
```

```
  + criarConsultaEmNomeDoPaciente(paciente, medico,\n    especialidade, horario):
    Consulta
  + cancelarConsulta(consulta, motivo: String): void
  + remarcarConsulta(consulta, novoHorario,\n    motivo: String): Consulta
}

class Administrador {
  + cadastrarMedico(dados): Medico
  + cadastrarSecretario(dados): Secretario
  + gerenciarEspecialidade(dados): Especialidade
}

class Especialidade {
  - id: Long
  - nome: String {unique}
  - descricao: String
  - status: StatusEspecialidade
  + ativar(): void
  + desativar(): void
}

class Agenda {
  - id: Long
  - diasAtendimento: Set<DiaSemana>
  - horaInicio: Time
  - horaFim: Time
  - duracaoSlot: Duration
  - antecedenciaMinima: Duration
  + gerarHorarios(data): List<Horario>
  + getHorariosDisponiveis(intervalo): List<Horario>
  + atualizarRegra(regra): void
}

class Horario {
  - id: Long
  - data: Date
  - horaInicio: Time
  - horaFim: Time
  - situacao: SituacaoHorario
  + estaDisponivel(): boolean
  + bloquear(motivo): void
  + liberar(): void
  + ocupar(): void
}

class Consulta {
  - id: Long
```

```
  - dataHora: DateTime
  - status: StatusConsulta
  - motivoCancelamento: String
  - confirmadaEm: DateTime
  - canceladaEm: DateTime
  - criadoEm: DateTime
  - atualizadoEm: DateTime
  + confirmar(por: Usuario): void
  + cancelar(por: Usuario, motivo: String): void
  + remarcar(novoHorario: Horario,\n    motivo: String): Consulta
  + iniciar(): void
  + encerrar(status): void
  + podeSerAlterada(): boolean
}

class Notificacao {
  - id: Long
  - evento: TipoEvento
  - canal: Canal
  - destinatario: String
  - enviadoEm: DateTime
  - statusEntrega: StatusEntrega
  - tentativas: int
  + registrarEnvio(): void
  + registrarFalha(): void
}

class Auditoria {
  - id: Long
  - perfil: Perfil
  - acao: String
  - entidade: String
  - entidadeId: Long
  - dataHora: DateTime
  - ip: String
  - userAgent: String
  + registrar(evento): void
}

' ===== RELACIONAMENTOS =====

' Generalização
Usuario <|-- Paciente
Usuario <|-- Medico
Usuario <|-- Secretario
Usuario <|-- Administrador

```

```
' Composição (o todo governa o ciclo de vida das partes)
Medico "1" *-- "1" Agenda : possui >
Agenda "1" *-- "1..*" Horario : gera >
Consulta "1" *-- "0..*" Notificacao : gera >

' Associações
Medico "0..*" -- "1..*" Especialidade : atende >
Horario "1" -- "0..*" Consulta : ocupa <
Paciente "1" -- "0..*" Consulta : titular de >
Medico "1" -- "0..*" Consulta : atende >
Especialidade "1" -- "0..*" Consulta : classifica >
Consulta "0..1" -- "0..1" Consulta : remarcadaDe >
Consulta "0..*" -- "0..1" Usuario : confirmadaPor >
Consulta "0..*" -- "0..1" Usuario : canceladaPor >
Usuario "1" -- "0..*" Auditoria : registra >

note bottom of Horario
  Histórico: várias consultas ao longo do tempo,
  no máximo 1 ativa
end note

@enduml
```

Fonte: Elaborado pelos autores (2026).